

Card: Criar Mock da API do CADSUS (Deploy no Render) e Implementar Integração no SISPNAIST

Objetivo

Desenvolver uma API simulada (Mock) do CADSUS e publicá-la no Render. Em seguida, implementar o cliente de integração no SISPNAIST para buscar os dados cadastrais do trabalhador (nome, data de nascimento, filiação, endereço) a partir do seu CNS ou CPF. A integração deve ser controlada via variáveis de ambiente, preparando o sistema para a futura virada de chave com o link oficial do DATASUS.

Contexto

Para o SISPNAIST gerenciar a saúde do trabalhador com precisão, precisamos garantir que o cadastro dessa pessoa esteja espelhado na base oficial do Ministério da Saúde. O CADSUS é o sistema estruturante responsável por isso.

Nesta atividade, você criará um serviço fictício que simula a resposta do CADSUS e fará o nosso sistema consumir esses dados. Dessa forma, quando o usuário digitar o CPF ou CNS na tela do SISPNAIST, o sistema buscará os dados oficiais e preencherá o formulário automaticamente.

Passo a Passo

1.Criação da API Mock do CADSUS:No mesmo repositório do Mock anterior (ou em um novo).

- Criar um *endpoint* específico para buscar os dados do usuário (ex: GET /api/v1/cadsus/usuarios/{cpf_ou_cns}).
- O *endpoint* deve retornar um JSON com os dados demográficos principais de um cidadão (nome, data de nascimento, nome da mãe, endereço, contatos).

2.Deploy do Mock no Render:Disponibilização pública.

- Atualizar o repositório no GitHub para que o Render faça o *deploy* automático da nova rota do CADSUS.
- Testar no navegador ou via Postman/Insomnia se a URL pública (ex: https://mock-ms-xyz.onrender.com/api/v1/cadsus/usuarios/12345678900) está retornando os dados corretamente.

3.Configuração de Variáveis de Ambiente:Arquivo .env.

- Adicionar no arquivo .env do SISPNAIST a variável MS_CADSUS_API_URL.
- Apontar para a rota raiz do serviço recém-hospedado no Render.

- **Atenção:** Nenhuma URL externa deve ser "chumbada" (*hardcoded*) diretamente no código-fonte.

4.Implementação do Cliente (SISPNAIST):Conexão e Regras de Negócio.

- Criar o serviço que fará a requisição HTTP.
- **Padrão Adapter:** Mapear o JSON de resposta da API do CADSUS para a entidade/modelo Trabalhador (ou Paciente) do banco de dados do SISPNAIST.
- **Tratamento de Exceções:**
 - Se o usuário não for encontrado no CADSUS (Erro 404), defina o comportamento do SISPNAIST (Ex: Limpar a tela, permitir o cadastro manual e emitir um alerta amigável).
 - Se a API cair ou der *Timeout*, exibir uma mensagem de erro clara para o usuário final, sem "quebrar" a tela.

✓ Critérios de Aceite

- [] O *endpoint* do CADSUS está funcional e acessível publicamente pela URL do Render.
- [] O SISPNAIST realiza a busca no CADSUS lendo a URL a partir do arquivo .env.
- [] A funcionalidade de "Buscar por CPF/CNS" está operando na interface do SISPNAIST (preenchendo os dados automaticamente em tela ou no *backend*).
- [] O código implementa o padrão **Adapter**, isolando os campos do Ministério da Saúde dos campos nativos do SISPNAIST.
- [] Cenários de erro (API indisponível, Cidadão não encontrado) estão devidamente tratados e repassam *feedback* ao usuário.

💡 Dicas e Guia de Apoio

1. Contrato (JSON) esperado para o Mock do CADSUS

Para agilizar seu trabalho, use este modelo como base para o retorno da sua API fictícia. Ele reflete a estrutura de dados que o Ministério da Saúde costuma trafegar no barramento do CADSUS:

JSON

```
{  
  "cns_definitivo": "701234567890123"  
  "cpf": "12345678900"  
  "nome_completo": "João Trabalhador da Silva"  
  "data_nascimento": "1985-04-12"  
  "nome_mae": "Maria da Silva"  
  "sexo": "M"
```

```
"nacionalidade": "Brasileira",
"raca_cor": "Parda",
"endereco": {
  "logradouro": "Rua das Indústrias",
  "numero": "100",
  "complemento": "Apt 202",
  "bairro": "Polo Industrial",
  "municipio": "Camaçari",
  "uf": "BA",
  "cep": "42800-000"
},
"contato": {
  "telefone_principal": "71999999999",
  "email": "joao.trabalhador@email.com"
}
```

2. A Mágica do Adapter (De/Para)

É muito provável que o banco de dados do SISPNAIST não chame o campo de `cns_definitivo`, mas sim apenas `cns`, e talvez `nome_completo` seja apenas `nome`. O seu código de integração deve ter uma função (o *Adapter*) que receba o JSON bruto acima e o transforme no nosso objeto interno antes de enviar para a tela ou salvar no banco. **Isso blinda o nosso sistema de futuras mudanças na API do Ministério da Saúde.**

3. UX e o "Cold Start" do Render

Como o plano gratuito do Render "dorme" após 15 minutos sem uso, a primeira chamada pode demorar alguns segundos. Aproveite essa latência para **implementar um *loading* (tela de carregamento ou "spinner")** na interface do SISPNAIST. Como essa busca é interativa, a interface não pode parecer travada. Mostre um aviso claro como: *"Consultando a base do Ministério da Saúde..."*.